

AI AGENT FRAMEWORKS

AI 에이전트 개발 프레임워크

LangChain · LangGraph · CrewAI 핵심 개념과 선택 가이드

From Chains to Graphs to Crews — Building Production-Grade Agents

LC

LangChain

LG

LangGraph

CA

CrewAI

+α

Others

AI 에이전트 프레임워크 지형도

추상화의 층위로 보는 프레임워크 — 어디에 무엇이 있는가

추상화 스택

애플리케이션 RAG 챗봇, 코딩 에이전트, 자동화 워크플로

역할 기반 협업 CrewAI · Agent-Task-Crew 추상화로 빠른 프로토타입

그래프 오케스트레이션 LangGraph · State-Node-Edge로 정밀 제어

컴포넌트 LangChain · LCEL · Tools · RAG · Messages

LLM API OpenAI · Anthropic · Google · Ollama · 등

주요 프레임워크

LangChain LLM 컴포넌트 조립의 표준. LCEL로 파이프라인 구성

LangGraph 그래프 기반 멀티 에이전트 오케스트레이션

CrewAI 역할 기반 협업, Crews + Flows 이중 구조

Google ADK Google Agent Development Kit, Gemini 친화

Claude Agent SDK Anthropic, MCP-서브에이전트 네이티브

왜 프레임워크인가 — Build vs Adopt

단일 LLM API 호출만으로는 풀기 어려운 문제들이 빠르게 늘고 있다

RAW SDK

직접 빌드 (Roll Your Own)

- ▶ 모델별로 **다른 SDK**를 학습 (OpenAI / Anthropic / Google)
- ▶ 도구 호출, 메시지 포맷, 스트리밍을 **매번 직접 구현**
- ▶ RAG, 메모리, 멀티 에이전트는 **전부 처음부터**
- ▶ 장점: 완전한 제어, 가벼움, 의존성 최소
- ▶ 한계: 반복되는 보일러플레이트, 빠른 변화 추적 비용

FRAMEWORK

프레임워크 채택 (Adopt)

- ▶ **통일된 추상화**로 LLM·도구·메모리를 한 인터페이스로
- ▶ **검증된 패턴**: ReAct, RAG, Supervisor 등을 즉시 사용
- ▶ **생태계**: 통합·관측·디버깅 도구가 함께 따라옴
- ▶ 장점: 빠른 프로토타입, 모델 교체 용이, 커뮤니티 베스트프랙티스
- ▶ 한계: 학습 곡선, 추상화 비용, 버전 변화에 민감

현실적 권장: 단순 호출은 SDK로, **멀티 단계·멀티 에이전트·상태가 필요해지면 프레임워크**로 전환.

LangChain Core Components — 공식 8종

"prebuilt agent architecture and integrations for any model or tool" · **built on top of LangGraph**



01

Agents

AUTONOMOUS LOOP

루프 안에서 **도구를 자율 호출**하는 LLM 실행 단위. create_agent로 10줄 미만 시작.



02

Models

UNIFIED INTERFACE

OpenAI · Anthropic · Google 등 **provider 통일 인터페이스**. 한 줄로 교체.



03

Messages

CONVERSATION UNIT

role + content. System/Human/AI/Tool + **multimodal** 블록(text/image).



NEW

Tools

EXTERNAL ACTIONS

@tool 데코레이터. **ToolRuntime**으로 state-store-context 접근.



05

Short-term Memory

IN-CONVERSATION

대화 세션 내 컨텍스트. MessagesState + **Checkpoint**으로 thread별 영속.



06

Streaming

REAL-TIME UPDATES

에이전트 실행 중 **토큰·중간 단계·도구 호출**까지 점진 노출. 4가지 mode.



07

Structured Output

TYPED RESPONSE

Pydantic 스키마로 **타입 안전 응답**. ToolStrategy / ProviderStrategy.



NEW

Middleware

EXECUTION HOOKS

에이전트 루프를 **가로채 제어**. 6개 + 빌트인(요약·HITL·PII-Fallback).

통일된 인터페이스 — Provider만 바꾸면 끝

`init_chat_model()` 한 줄로 OpenAI · Anthropic · Google 등을 동일하게 호출

OPENAI

GPT 모델 호출

```
from langchain.chat_models import init_chat_model

model = init_chat_model(
    "gpt-5-mini",
    model_provider="openai"
)

result = model.invoke("랭체인이 뭔가요?")

# result 타입 → AIMessage
print(result.content)
```

ANTHROPIC

Claude 모델 호출

```
from langchain.chat_models import init_chat_model

model = init_chat_model(
    "claude-sonnet-4",
    model_provider="anthropic"
)

result = model.invoke("랭체인이 뭔가요?")

# result 타입 → AIMessage (동일)
print(result.content)
```

공식 권장(Provider Prefix): `init_chat_model("anthropic:claude-sonnet-4-6", temperature=0.7)` 한 줄로 가능.

메서드 매트릭스: `invoke` · `stream` · `batch` · `ainvoke` · `astream_events` 모두 자동 지원.

Messages — 채팅 모델의 기본 단위

단순 문자열을 넘어, role과 content를 가진 구조화된 대화 단위

5가지 메시지 타입

SystemMessage

역할·정책 지시 — 모델 페르소나 설정

HumanMessage

사용자 입력 — 질문·요청

AIMessage

모델 응답 — invoke() 결과의 기본 타입

ToolMessage

도구 실행 결과 — Tool Calling 응답

*MessageChunk

스트리밍 단위 — 점진적 응답

임포트: from langchain.messages import HumanMessage (1.0+) 또는
langchain_core.messages

멀티모달: content 자리에 [{"type": "text", ...}, {"type": "image", ...}] 블록
목록 리스트

멀티턴 대화 예시

SYSTEM

당신은 친절한 한국어 비서입니다.

HUMAN

파이썬 리스트를 정렬하는 법은?

AI

list.sort() 또는 sorted(list)를 사용합니다...

HUMAN

역순으로 정렬하려면?

AI

reverse=True 옵션을 추가하시면 됩니다.

대화 = 메시지 리스트 · 모델 입력은 항상 메시지의 시퀀스이며, 각 턴이 누적된다.

PromptTemplate — 프롬프트를 코드로

문자열 포매팅을 넘어 객체 기반 재사용·테스트·버전 관리

4가지 템플릿 타입

PromptTemplate
기본 단일 프롬프트

ChatPromptTemplate
멀티 메시지 (System+Human)

FewShotPromptTemplate
예제 기반 학습

MessagesPlaceholder
멀티턴 메시지 누적 슬롯

4가지 생성법: from_template / 생성자 / load_prompt(YAML) / partial_variables

템플릿 → 변수 치환 → 프롬프트

```
from langchain_core.prompts import PromptTemplate

template = PromptTemplate.from_template(
    "당신은 친절한 AI입니다.\n질문: {question}\n답변:"
)

prompt = template.invoke({"question": "랭체인이 뭐죠?"})
```



"당신은 친절한 AI입니다.
질문: 랭체인이 뭐죠?
답변:"

중요: PromptTemplate은 Runnable — LCEL의 | 파이프로 모델·파서와 바로 연결된다.

1.0 진화: 에이전트에서는 system_prompt(고정) 또는 @dynamic_prompt(미들웨어, 상태 기반) 패턴이 표준

Structured Output — 텍스트를 타입으로

두 트랙: **OutputParser**(LCEL용) · **ToolStrategy/ProviderStrategy**(에이전트용)

TRACK 1 · LCEL

OutputParser 4종 (자주 쓰는 것)

StrOutputParser

JsonOutputParser

PydanticOutputParser

CommaSeparated

용도: prompt | model | parser 파이프 끝에 붙여 자유 텍스트 응답을 즉시 객체로.

TRACK 2 · AGENT

response_format 전략 2종

ToolStrategy

모든 모델 지원 (도구 호출 활용). 폴백 가능, 호환성 우선.

ProviderStrategy

OpenAI/Anthropic 네이티브 구조화 출력. 정확도·성능 우선.

용도: create_agent(..., response_format=ToolStrategy(Schema)) — 에이전트 최종 응답이 스키마 객체로 보장.

실전 — Pydantic 스키마 + 에이전트 결합

```
from langchain.agents import create_agent
from langchain.agents.structured_output import ToolStrategy
from pydantic import BaseModel

class Movie(BaseModel):
    title: str; year: int; rating: float

agent = create_agent("anthropic:claude-sonnet-4-6", tools=[search],
    response_format=ToolStrategy(Movie)) # 응답이 Movie로 보장
```


Runnable + LCEL — 파이프로 조립하는 LLM 컴포넌트

Unix 파이프처럼 자연스러운 데이터 흐름. 모든 컴포넌트가 같은 인터페이스를 갖는다.

✗ BEFORE

절차적 함수 호출 (강결합)

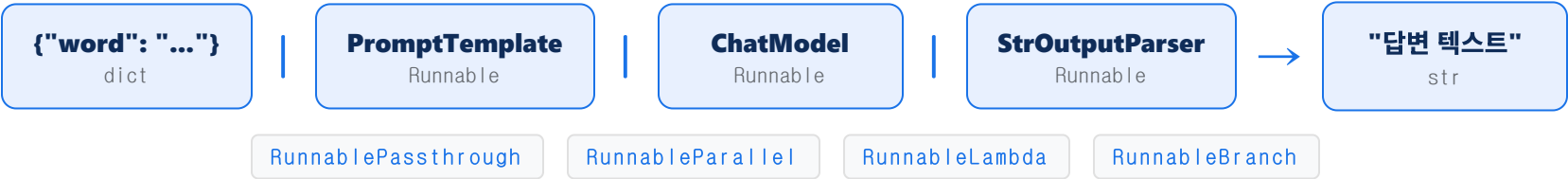
```
prompt = generate_prompt(input)
response = llm.generate(prompt)
parsed = parse_response(response)
# 컴포넌트 교체 시 코드 수정 필요
```

✓ AFTER (LCEL)

파이프로 조립 (느슨한 결합)

```
chain = prompt | model | parser
result = chain.invoke({"input": input})
# 컴포넌트 교체 = 한 곳만 변경
```

데이터 흐름 — 같은 인터페이스(involve-stream-batch)로 연결되는 Runnable



스코프: 단순 변환 파이프(prompt|model|parser) → LCEL · 도구 루프·상태·분기 → create_agent(LangGraph 런타임)

create_agent — LangChain 1.0의 진입점

"빌드된 에이전트 아키텍처" 한 줄 호출 — model · tools · system_prompt만 있으면 시작

기본 예제

가장 간단한 에이전트 만들기

```
from langchain.agents import create_agent
from langchain.tools import tool

@tool
def get_weather(city: str) -> str:
    # LLM은 이 docstring으로 도구를 선택
    return f"{city}는 맑음"

agent = create_agent(
    model="anthropic:claude-sonnet-4-6",
    tools=[get_weather],
    system_prompt="너는 친절 한 비서야",
)

agent.invoke({"messages": [
    {"role": "user", "content": "서울 날씨"}
]})
```

핵심: 내부적으로 **LangGraph 런타임**으로 컴파일됨 — invoke-stream-체크포인트 자동 지원

6대 인자

model	provider prefix 문자열 또는 ChatModel
tools	@tool 함수 리스트 또는 ToolNode
system_prompt	고정 페르소나·정책 문자열
response_format	ToolStrategy / ProviderStrategy로 구조화 출력
state_schema	커스텀 State 스키마 (TypedDict / Pydantic)
middleware	실행 루프 가로채기 혹 리스트

Tools — @tool 데코레이터 + ToolRuntime

함수 → 도구 변환은 한 줄. ToolRuntime으로 state-store-context까지 접근.

기본: @tool 데코레이터

```
from langchain.tools import tool

@tool
def get_weather(city: str) -> str:
    """도시의 날씨를 반환""" # ← LLM의 도구 선택 기준
    return f"{city}는 맑음"

# 1.0 표준: create_agent로 직접 전달
agent = create_agent(model, tools=[get_weather])
```

반환값 3종 (1.0 표준)

str — 단순 텍스트 결과

dict — 구조화된 결과

Command — state 업데이트 + 흐름 제어

고급: ToolRuntime으로 컨텍스트 접근

```
from langchain.tools import tool, ToolRuntime

@tool
def save_pref(key: str, value: str,
              runtime: ToolRuntime) -> str:
    """사용자 선호 저장"""
    runtime.store.put(("user",), key, value)
    runtime.stream_writer(f"saved {key}")
    return "ok"
```

ToolRuntime 6대 속성

state 현재 그래프 상태	store 장기 메모리 KV
context 실행 메타데이터	stream_writer 실시간 진행 푸시
tool_call_id 호출 식별자	execution_info 실행 정보

Middleware — 에이전트 루프를 가로채다

"Control and customize agent execution at every step" — 1.0의 핵심 신기능

6개 훅 (Hooks)

@before_model	모델 호출 직전 (입력 가공)
@wrap_model_call	모델 호출 감싸기 (재시도/Fallback)
@after_model	모델 응답 후 (검증/필터)
@wrap_tool_call	도구 호출 감싸기 (승인/캐싱)
@dynamic_prompt	상태 기반 시스템 프롬프트 동적 생성
@before_agent	에이전트 첫 실행 전 (초기화)

빌트인 미들웨어 6종

SummarizationMiddleware 긴 대화 자동 요약 (토큰 절약)	HumanInTheLoopMiddleware 도구 실행 전 사용자 승인
PIIDetectionMiddleware 개인정보 자동 마스킹	ModelFallbackMiddleware 주 모델 장애 시 보조 모델 자동 전환
ToolRetryMiddleware 도구 실패 시 재시도	ModelCallLimitMiddleware 호출 횟수 상한 (비용 제어)

```
from langchain.agents.middleware import SummarizationMiddleware

agent = create_agent(
    model, tools,
    middleware=[SummarizationMiddleware(max_tokens=4000)]
)
```

RAG — Retriever as Tool

전통 LCEL 체인 → 1.0 권장: **retriever**를 에이전트의 도구로 노출



전통 — LCEL RAG

prompt | model 체인에 retriever 결합

```
retriever = vectorstore.as_retriever(
    search_type="similarity", k=3)

chain = {
    "context": retriever,
    "question": RunnablePassthrough()
} | prompt | llm

chain.invoke("초보자가 배우기 좋은 언어?")
```

단순 RAG · 단발성 Q&A에 적합

권장 — Retriever as Tool

에이전트가 필요할 때 검색 호출

```
@tool
def search_docs(query: str) -> str:
    """사내 문서 의미 기반 검색"""
    docs = retriever.invoke(query)
    return "\n".join(d.page_content for d in docs)

agent = create_agent(
    "anthropic:claude-sonnet-4-6", tools=[search_docs])
```

장점: 다중 도구 결합·재시도·여러 번 검색 가능

Memory — Short-term(State) vs Long-term(Store)

대화 내(thread) 기억과 사용자/세션 횡단(namespace) 기억의 분리

SHORT-TERM

State (Checkpoint)

In-conversation context · thread 단위

- ▶ **대상:** 현재 대화 세션의 메시지·중간 결과
- ▶ **저장:** Checkpointer (InMemory / SQLite / Postgres)
- ▶ **키:** thread_id
- ▶ **유효 범위:** 같은 thread 안에서만

```
from langgraph.checkpoint.memory import InMemorySaver

agent = create_agent(
    model, tools,
    checkpointer=InMemorySaver()
)
config = {"configurable": {"thread_id": "alice-1"}}
agent.invoke(input, config)
```

LONG-TERM

Store

Cross-session memory · namespace 단위

- ▶ **대상:** 사용자 프로필·선호·과거 결정
- ▶ **저장:** Store (InMemory / Postgres + 시맨틱 검색)
- ▶ **키:** (namespace,) 튜플
- ▶ **유효 범위:** 모든 thread 횡단

```
from langchain.tools import tool, ToolRuntime

@tool
def save_pref(key: str, value: str,
              runtime: ToolRuntime) -> str:
    """사용자 선호 저장"""
    runtime.store.put(("users", "alice"),
                      key, value)
    return "saved"
```

Multi-Agent 5 패턴 — 공식 카탈로그

에이전트를 결합하는 다섯 가지 표준 방식 — Subagents · Handoffs · Skills · Router · Custom

PATTERN 01



Subagents

에이전트를 **도구**로 노출하여 호출. 메인 컨텍스트를 깨끗이 유지.

단일 요청 분해

PATTERN 02



Handoffs

에이전트가 다른 에이전트에게 **제어 이전**. Tool call로 표현.

반복 요청 (40-50% 토큰 절감)

PATTERN 03



Skills

재사용 가능한 **능력 모듈**. 동적으로 로드해 에이전트에 부여.

반복 요청 (능력 재사용)

PATTERN 04



Router

분류기 LLM이 요청을 보고 **전문 에이전트**로 라우팅.

도메인 분기 (분류→처리)

PATTERN 05



Custom Workflow

위 패턴으로 안 되면 **LangGraph**로 직접 그래프 작성.

대규모 컨텍스트·정밀 제어

최단 Subagent 예제 — 에이전트를 도구로

```
research = create_agent(model, tools=[search_web], name="researcher")
writer = create_agent(model, tools=[research.as_tool()], name="writer") # writer가 researcher를 호출
```

왜 LangGraph인가 — Chain에서 Graph로

"low-level orchestration framework for **stateful, long-running agents**" — 공식 정의

X LIMITATION

LangChain Agent의 한계

- ▶ **단방향 흐름** — 체인은 본질적으로 선형
- ▶ **분기 표현 어려움** — if-else를 코드 밖으로 빼기 힘들
- ▶ **루프 부자연스러움** — 재시도·반복 로직이 복잡
- ▶ **병렬 실행 한계** — 동시 작업 표현 어려움
- ▶ **상태 가시성 부족** — 디버깅·감사 어려움

✓ LANGGRAPH

그래프 모델로 해결

- ▶ **노드 + 엣지**로 워크플로 명시적 모델링
- ▶ **조건부 라우팅**으로 동적 분기를 자연스럽게
- ▶ **사이클(루프)** 네이티브 지원 — 재시도·ReAct
- ▶ **병렬 실행** 자동 — 의존성 없는 노드는 동시 실행
- ▶ **State 중심** — 모든 노드가 같은 상태를 공유·관측

PRODUCTION USERS · 글로벌 대기업의 프로덕션 환경 채택

Klarna.

</> Replit

Uber

elastic

in LinkedIn

— LangChain.inc 공식 발표

LangGraph 핵심 3요소 — State · Node · Edge

모든 LangGraph 워크플로는 이 세 가지 추상화로 구성된다



SHARED DATA

State

워크플로 전체에서 공유되는 **중앙 데이터 저장소**. 각 노드가 읽고 쓸 수 있다. **TypedDict** 또는 **Pydantic** 모델로 정의하여 타입 안정성을 보장한다.

TypedDict

Pydantic

MessagesState

Reducer (add_messages)



EXECUTION UNIT

Node

그래프의 기본 **실행 단위**. 함수 또는 에이전트로 구현된다. **현재 상태를 입력**받아 작업을 수행하고, **업데이트된 상태**를 반환한다.

function

agent

ToolNode



FLOW CONTROL

Edge

노드 간 **연결**로 실행 흐름을 제어한다. **일반 에지**는 항상 같은 경로, **조건부 에지**는 상태에 따라 동적으로 다음 노드를 결정한다.

add_edge

add_conditional_edges

공식 멘탈 모델: "Nodes do the work, edges tell what to do next." · 동시 실행 단위 = **Superstep**

Hello LangGraph — 5분 안에 만드는 첫 그래프

StateGraph → add_node → add_edge → compile → invoke

표준 4단계 패턴

```
from langgraph.graph import StateGraph, START, END
from pydantic import BaseModel, Field

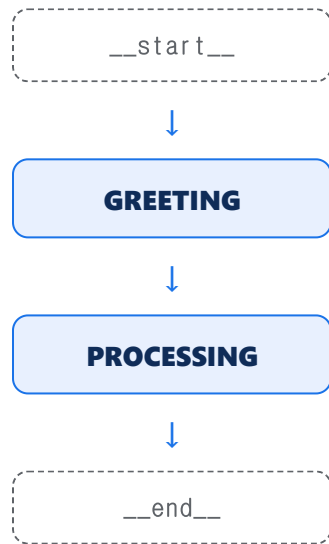
# 1. State 정의
class GraphState(BaseModel):
    name: str = Field(default="")
    greeting: str = Field(default="")

# 2. Node 함수 정의
def generate_greeting(state):
    return {"greeting": f"안녕, {state.name}!"}

# 3. 그래프 조립
workflow = StateGraph(GraphState)
workflow.add_node("greeting", generate_greeting)
workflow.add_edge(START, "greeting")
workflow.add_edge("greeting", END)

# 4. 컴파일 + 실행
app = workflow.compile()
result = app.invoke({"name": "민수"})
# {"name": "민수", "greeting": "안녕, 민수!"}
```

실행 흐름 시각화



시각화: `app.get_graph().draw_mermaid_png()`으로 그래프를 PNG로 저장 가능

조건부 라우팅 — LLM이 길을 선택한다

add_conditional_edges로 런타임 동적 분기 (예: 감정 분석 챗봇)

코드 패턴

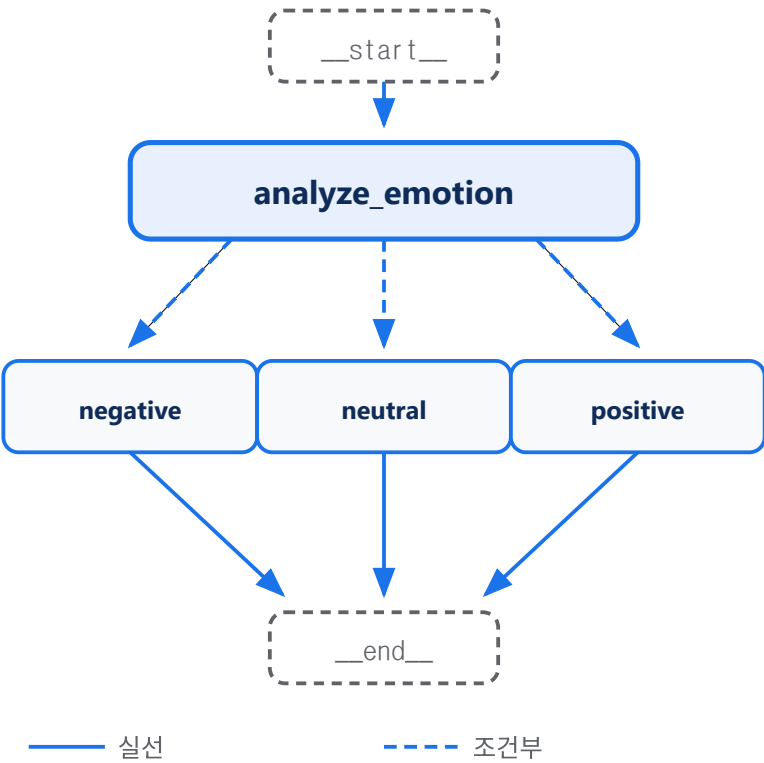
```
from typing import Literal

# 라우팅 함수: state → 다음 노드 이름
def route_by_emotion(state) -> Literal[
    "positive", "negative", "neutral"]:
    return state.emotion

# 조건부 엣지 등록 (path_map 권장)
workflow.add_conditional_edges(
    "analyze_emotion",
    route_by_emotion,
    {"positive": "praise_node",
     "negative": "soothe_node",
     "neutral": "general_node"}
)
```

핵심: 라우팅 함수는 Literal[...] 반환 타입 + path_map으로 시각화/디버깅 명확화

감정 분석 챗봇 그래프



Cycles — 그래프에 루프 만들기

조건부 엣지로 이전 노드로 돌아가면 곧 사이클 (예: 숫자 맞추기 게임)

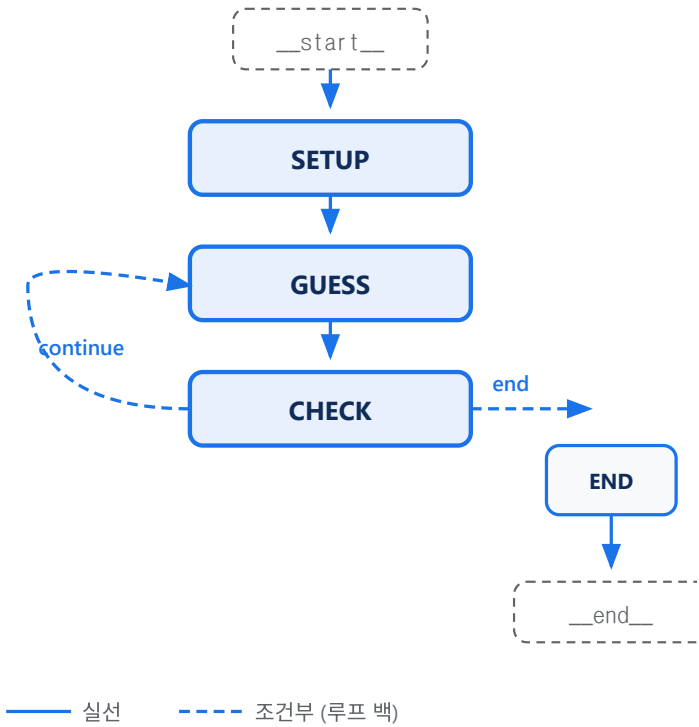
루프 패턴

```
# 계속할지 / 끝낼지 결정
def route_game(state):
    if state.attempts >= 5 or state.correct:
        return "end"
    return "continue"

# 조건부 엣지가 이전 노드로 돌아가면 루프
workflow.add_conditional_edges("check", route_game, {
    "continue": "guess", # ← 루프 백
    "end": END
})
```

- 활용
- 재시도 로직
- 대화형 반복
- 조건 만족까지 반복
- ReAct 사이클

숫자 맞추기 게임 그래프



Checkpoint — 단 3줄로 영속성·복구

대화 이력을 자동으로 저장/복원하고, 같은 thread_id로 세션을 이어갈 수 있다

3단계 사용법

- 1 체크포인트 설정 — 저장소 인스턴스 생성
- 2 compile에 전달 — workflow.compile(checkpointer=...)
- 3 thread_id로 세션 식별 — config에 configurable.thread_id

```
from langgraph.checkpoint.memory import InMemorySaver

checkpointer = InMemorySaver()
app = workflow.compile(checkpointer=checkpointer)

config = {"configurable": {"thread_id": "user_123"}}
result = app.invoke({"user_message": msg}, config)

# 다음 호출 시 같은 thread_id면 이전 상태 자동 로드
result2 = app.invoke({"user_message": "이전 답 다시"}, config)
```

자동 동작: 노드 실행 전후 자동 체크포인트 → 이전 상태와 새 입력 자동 병합

체크포인트 종류

- InMemorySaver**
메모리 기반 · 휘발성 · 개발/테스트용
- SqliteSaver**
SQLite 파일 · 단일 노드 · 소규모 프로덕션
- PostgresSaver**
PostgreSQL · 멀티 노드 · 대규모 프로덕션
- BaseCheckpointSaver**
추상 클래스 · 커스텀 백엔드 구현 가능 (Redis 등)

활용 시나리오: 챗봇 대화 이력 · 장기 실행 워크플로 복구 · HITL 중단점 재개 · 감사 로그

병렬 실행 — Fan-out / Fan-in

의존성 없는 노드는 자동으로 병렬 실행 — 코드 없이도 성능이 따라온다

패턴 & 성능

FAN-OUT

한 노드(Coordinator)가 여러 노드로 작업을 분기

FAN-IN

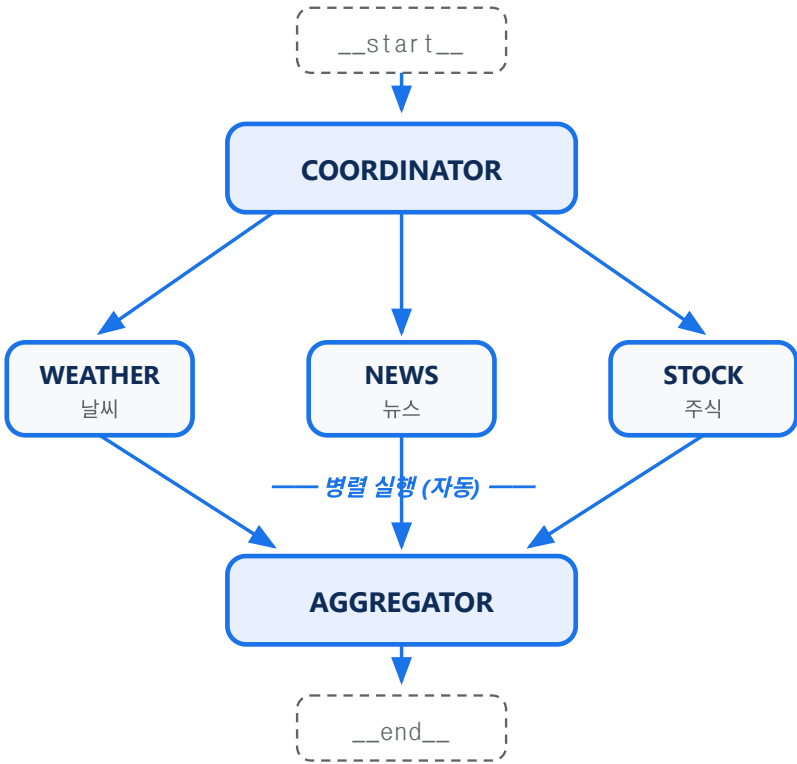
여러 노드의 결과를 한 집계자(Aggregator)가 통합

실행 시간 비교

순차	~ 7.0s
병렬	~ 2.3s

핵심: 같은 단계의 노드들이 독립적이면 LangGraph가 자동 병렬화한다.

대시보드 그래프 — 날씨·뉴스·주식 동시 조회



동적 fan-out: `Send("worker", {"item": x}) for x in items` — Orchestrator-Worker 패턴

ToolNode — ReAct 패턴을 prebuilt로

LLM ↔ 도구 사이의 사이클을 단 몇 줄로 구현 (계산기·날씨·환율 등)

표준 ReAct 패턴

```
from langgraph.prebuilt import ToolNode
from langgraph.graph import MessagesState

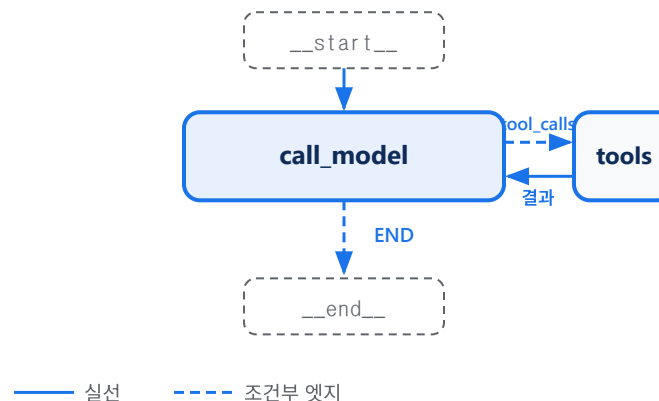
# 1. 도구 정의 (LangChain @tool 그대로)
tools = [calculator, get_weather, currency_converter]

# 2. ToolNode (prebuilt) + 도구 바인딩
tool_node = ToolNode(tools)
model_with_tools = model.bind_tools(tools)

# 3. 분기 함수: tool_calls가 있으면 tools로
def should_continue(state: MessagesState):
    if state["messages"][-1].tool_calls:
        return "tools"
    return END

# 4. 그래프 조립 - 사이클
workflow.add_conditional_edges("call_model",
    should_continue, ["tools", END])
workflow.add_edge("tools", "call_model")
```

실행 흐름



ReAct = Reasoning + Acting

- R** 모델이 어떤 도구를 쓸지 **추론**
- A** 도구 호출(tool call) **실행**
- O** 결과(observation)를 다시 모델에게
- 🔄** 충분할 때까지 반복

Streaming — 실시간 진행 상태 노출

stream_mode 4종 (+ 보조 3종) · v1.1 통합 포맷 version="v2"

stream_mode 4종 (핵심)

values	전체 상태를 매 단계 스냅샷으로 (디버깅·감사용)
updates	각 노드의 변경분(델타)만 (가장 흔한 선택)
messages	LLM의 토큰 단위 스트리밍 (UX 핵심)
custom	노드 내부에서 직접 푸시 — get_stream_writer()

checkpoints tasks debug

실전 — get_stream_writer로 진행률 푸시

```
from langgraph.config import get_stream_writer

def slow_node(state):
    writer = get_stream_writer()
    writer({"status": "fetching..."})
    # ... 긴 작업 ...
    writer({"status": "done"})
    return {"result": "..."}

# 호출 측: 여러 모드 동시 + v2 포맷
for chunk in graph.stream(
    inputs,
    stream_mode=["updates", "custom"],
    version="v2"):
    print(chunk["type"], chunk["data"])
```

v1.1 통합 포맷: 모든 chunk가 {"type", "ns", "data"} 일관 구조 — UI 통합 단순화

Human-in-the-Loop · Subgraph — 협업과 모듈화

사람의 개입이 필요한 지점, 그리고 그래프를 노드처럼 쓰는 모듈화

HITL

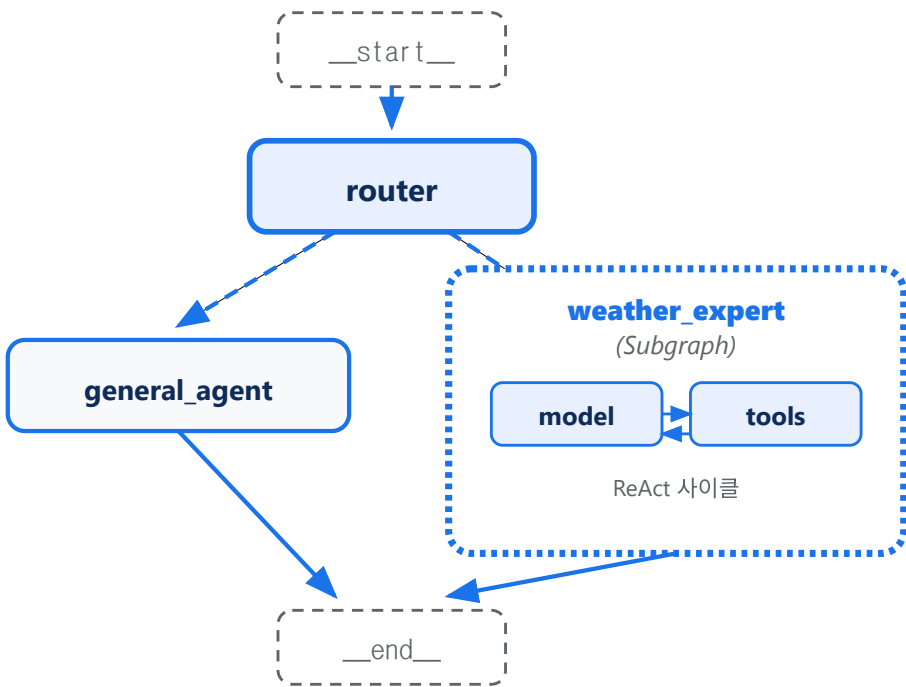
Human-in-the-Loop 4단계

- 1 Breakpoint**
AI가 인간 개입이 필요한 지점 인식
- 2 Context**
결정에 필요한 정보를 인간에게 제공
- 3 Feedback**
구조화된 입력으로 피드백 수집
- 4 Resume**
인간 결정에 따라 그래프 실행 재개

1.0+ 표준 API: `interrupt({"질문": ...}) + graph.invoke(Command(resume=값), config)` · 체크포인트 필수

SUBGRAPH

Subgraph — 그래프를 노드처럼



장점: 모듈화 · 독립 테스트 · 재사용 — Supervisor 패턴(메인이 전문가들 조율)의 기초

CrewAI — Crews + Flows의 이중 구조

자율적 협업(Crews)과 결정론적 제어(Flows)를 한 프레임워크에서

AXIS 1

Crews

"More Agency" — 자율적 협업

- ▶ 역할 기반 위임 — Agent가 다른 Agent에게 작업 전달
- ▶ 동적 의사결정 — LLM 추론으로 다음 단계 결정
- ▶ 협상·협력 — 에이전트 간 상호작용
- ▶ 빠른 프로토타입 — ~20줄로 멀티 에이전트 구성

AXIS 2

Flows

"Finer Precision" — 결정론적 오케스트레이션

- ▶ 이벤트 기반 트리거 — @start , @listen , @router
- ▶ 조건 분기·루프 — Python 코드로 명시적 제어
- ▶ State 관리 — Pydantic 기반 단일 진실의 원천
- ▶ 영속성 — @persist 데코레이터

두 축의 통합

Flows can call Crews ↔ Crews can be steps in Flows

"결정론적 워크플로 안에서 자율적 에이전트 팀을 호출" — 두 패러다임의 조합으로 실무 요구사항을 만족

4대 핵심 컴포넌트 — Agent · Task · Crew · Process

역할 기반 멀티 에이전트 시스템을 빠르게 구성하는 4-요소 모델

1

Agent
EXECUTION UNIT

역할을 가진 자율 실행 단위. role + goal + backstory로 정체성을 부여

role

goal

backstory

tools

reasoning ★

multimodal ★

inject_date ★

2

Task
WORK UNIT

개별 작업 단위. 무엇을 할지(description)와 무엇을 받을지(expected_output) 명시

description

expected_output

agent

context

3

Crew
TEAM CONTAINER

에이전트 팀과 워크플로의 컨테이너. 실행을 관장하는 최상위 객체

agents

tasks

process

memory

manager_llm

4

Process
EXECUTION STRATEGY

실행 전략. **Sequential**은 순차, **Hierarchical**은 Manager LLM이 동적 위임

sequential

hierarchical

최소 실행 가능 예제

```
from crewai import Agent, Task, Crew, Process

agent = Agent(role="Researcher", goal="AI 트렌드 조사", backstory="...")
task = Task(description="2026 AI 에이전트 동향", expected_output="보고서", agent=agent)
crew = Crew(agents=[agent], tasks=[task], process=Process.sequential)
result = crew.kickoff(inputs={"topic": "멀티 에이전트"})
```

CrewAI Flow 데코레이터 4종

이벤트 기반 워크플로 — Pydantic State + 트리거 체인으로 결정론적 제어

@start()

진입점 (Entry Point)

Flow의 시작점을 표시. 여러 개를 둘 수 있고, 모두 병렬 실행됨.

→ 첫 단계 실행 후 자동으로 listener 호출

@listen(prev)

리스너 (Listener)

지정된 이전 메서드가 완료되면 트리거. 이전 단계의 반환값을 인자로 받음.

→ AND/OR 결합도 가능 (and_, or_)

@router(prev)

라우터 (Router)

조건을 평가해 다음 분기 키를 문자열로 반환. 다른 listener가 그 키로 트리거됨.

→ if-else를 데코레이터로 표현

@persist

영속화 (Persistence)

State를 SQLite에 자동 저장. 클래스 또는 메서드 레벨로 적용 가능. 중단/재개 지원.

→ Long-running flow의 핵심

다음 슬라이드: 4가지 데코레이터를 모두 사용한 단일 흐름 전체 코드 예제

Flow 단일 흐름 전체 예제 — 콘텐츠 생성

@persist + @start → @listen → @router → @listen("키") 분기 흐름

```
from crewai.flow.flow import Flow, start, listen, router
from crewai.flow.persistence import persist
from pydantic import BaseModel

class ContentState(BaseModel):
    topic: str = ""
    quality_score: int = 0

@persist # 클래스 레벨: 전체 State 자동 저장 (SQLite)
class ContentFlow(Flow[ContentState]):

    @start() # 진입점 - Flow 실행 시 가장 먼저
    def gather_topic(self):
        self.state.topic = "AI Agents 2026"

    @listen(gather_topic) # gather_topic 완료 시 트리거
    def evaluate_quality(self):
        self.state.quality_score = 85

    @router(evaluate_quality) # 평가 결과 기반 분기
```

CrewAI Memory — 통합 API (4종 → 단일)

2026 변화: 4종 분리 모델 → 단일 Memory 클래스 (LanceDB(랜스디비) 기반, 적응형 recall)

변화의 핵심

PAST · 과거 4종 분리

Short-term

Long-term

Entity

Contextual

NOW · 단일 Memory 통합

recency · semantic · importance 가중치로 자동 조합. 4종은 내부 구현으로 흡수.

```
from crewai import Crew

# 가장 간단한 활성화
crew = Crew(
    agents=[...], tasks=[...],
    memory=True
)
```

3가지 Recall 가중치

```
from crewai import Crew, Memory

memory = Memory(
    recency_weight=0.4, # 최신성
    semantic_weight=0.4, # 의미 유사도
    importance_weight=0.2 # 중요도 (LLM 자동)
)
crew = Crew(memory=memory, ...)
```

recency_weight 최신 기억일수록 가중 (지수 감쇠)

semantic_weight 현재 작업과 의미적으로 유사한 기억

importance_weight LLM이 자동 추론한 중요도 점수

CrewAI vs LangGraph — 언제 무엇을 쓸까

두 프레임워크의 추상화 모델 차이와 실무 적용 가이드

비교 항목	CrewAI	LangGraph
추상화 모델	역할 기반 에이전트 팀	상태 그래프 (노드 + 엣지)
코드량 (PoC)	~ 20줄	~ 60줄+
학습 곡선	낮음 (직관적)	중간-높음 (그래프 사고)
제어 정밀도	중간 (Flows로 보강)	높음 (모든 전이 명시)
상태 관리	In-memory 기본, @persist 확장	체크포인팅 시스템 성숙
협업 스타일	자율적 위임 + 협상	명시적 라우팅
HITL	@human_feedback (1.8+)	interrupt() + Command(resume=)
분기 제어	@router · Process.hierarchical	add_conditional_edges
MCP 통합	네이티브 Agent(mcps=[...])	Tool로 wrap 필요
적합한 상황	콘텐츠 생성, 리서치 팀, 빠른 PoC	복잡한 분기, 장기 실행, 엄격한 감사

한 줄 가이드: 속도가 우선이면 CrewAI, 엄격한 상태 제어가 필수면 LangGraph. 두 프레임워크 모두 LangChain 호환이라 점진적 마이그레이션이 가능하다.

프레임워크 선택 가이드

단일 정답은 없다 — 워크플로의 복잡도와 제어 요구에 맞춰 조합하라



단순 LLM 워크플로

단일 호출 · RAG 챗봇 · 프롬프트 체이닝
예: 문서 Q&A, 요약, 분류

RECOMMENDED

LangChain (LCEL)

또는 LLM SDK 직접 사용

↗ 홈

↗ GitHub

↗ Docs



멀티 에이전트 협업

역할 기반 팀 · 빠른 프로토타이핑 · 콘텐츠/리서치 자동화
예: 작가+편집자+검수자 팀

RECOMMENDED

CrewAI

~20줄로 시작 가능, 직관적

↗ 홈

↗ GitHub

↗ Docs



복잡한 분기·루프·HITL

상태 정밀 제어 · 장기 실행 · 감사 로그 · 분기 다중화
예: 자율 코딩, 워크플로 자동화

RECOMMENDED

LangGraph

결정론적 실행, 체크포인트링 성숙

↗ 홈

↗ GitHub

↗ Docs

결론 — 추상화의 위계로 프레임워크를 보자

KEY TAKEAWAYS

- 1 **LangChain**은 **컴포넌트 조립의 표준**. 모델·도구·메모리·검색을 통일된 인터페이스로 묶고 **LCEL**로 파이프 조립한다.
- 2 **LangGraph**는 체인의 한계를 넘어 **분기·루프·병렬·HITL**을 그래프로 모델링. State + Node + Edge로 모든 워크플로를 표현한다.
- 3 **CrewAI**는 역할 기반 협업으로 **빠른 프로토타입**에 강하고, Flows로 정밀 제어까지 보강할 수 있다.
- 4 선택은 **배타적이지 않다** — LangChain 위에 LangGraph를, LangGraph로 CrewAI 워크플로를 호출하는 식으로 **계층 조합**이 가능하다.

THE BIG PICTURE

프레임워크는 언어다.
풀려는 문제의 모양에 맞는 언어를 골라라.

실무 권장 순서

- ① LLM SDK로 빠르게 검증
- ② LangChain으로 컴포넌트화
- ③ 멀티 에이전트는 CrewAI로 PoC
- ④ 정밀 제어 필요 시 LangGraph로 견고화

참고자료 — 공식 문서 12선

LANGCHAIN · OVERVIEW LangChain Python Docs Core 8종 · create_agent 출발점 ↗ docs.langchain.com/oss/python/langchain	LANGCHAIN · MIDDLEWARE Middleware (1.0 핵심) 6종 + 빌트인 6종 (요약·HITL·PII-Fallback) ↗ .../langchain/middleware/overview	LANGCHAIN · MULTI-AGENT Multi-Agent 5패턴 Subagents · Handoffs · Skills · Router ↗ .../langchain/multi-agent	LANGCHAIN · MEMORY Long-term Memory (Store) Short-term(State) vs Long-term(Store) ↗ .../langchain/long-term-memory
LANGGRAPH · OVERVIEW LangGraph Docs State · Node · Edge · stateful agents ↗ docs.langchain.com/oss/python/langgraph	LANGGRAPH · STREAMING Streaming (4 modes) get_stream_writer · v2 통합 포맷 ↗ .../langgraph/streaming	LANGGRAPH · PERSISTENCE Checkpoint + Time Travel Durable Execution · get_state_history ↗ .../langgraph/persistence	LANGGRAPH · GRAPH API Send · Reducer · Superstep 동적 fan-out · add_messages reducer ↗ .../langgraph/use-graph-api
CREWAI · OVERVIEW CrewAI Docs Crews + Flows · Agent · Task · Process ↗ docs.crewai.com	CREWAI · FLOWS Flows + 데코레이터 4종 @start · @listen · @router · @persist ↗ docs.crewai.com/en/concepts/flows	CREWAI · MEMORY Memory (통합 API) recency · semantic · importance 가중치 ↗ docs.crewai.com/en/concepts/memory	REPO · GITHUB crewAI GitHub 45,900+ ★ · v1.10+ (2026-03) ↗ github.com/crewAIInc/crewAI